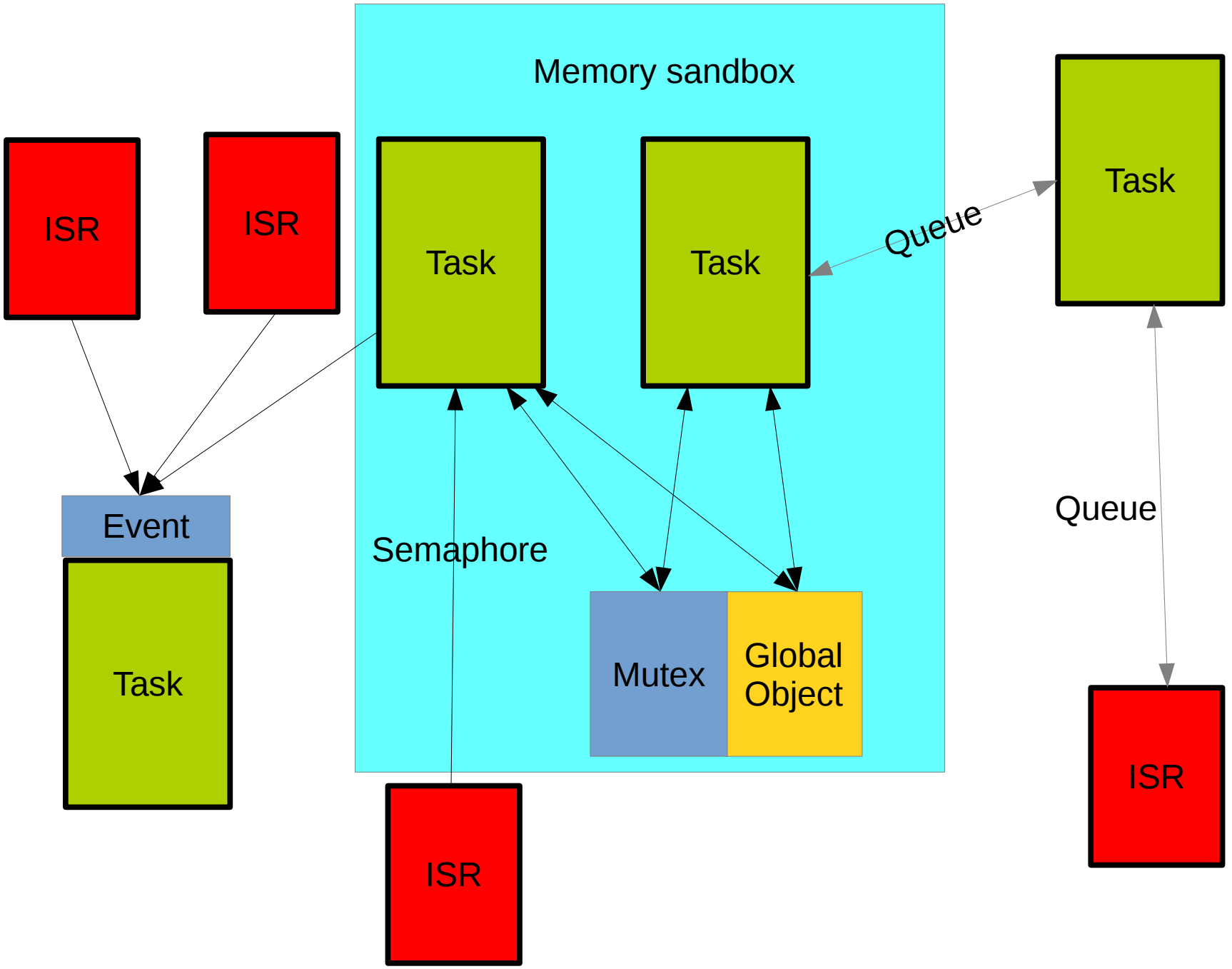
Lecture Overview

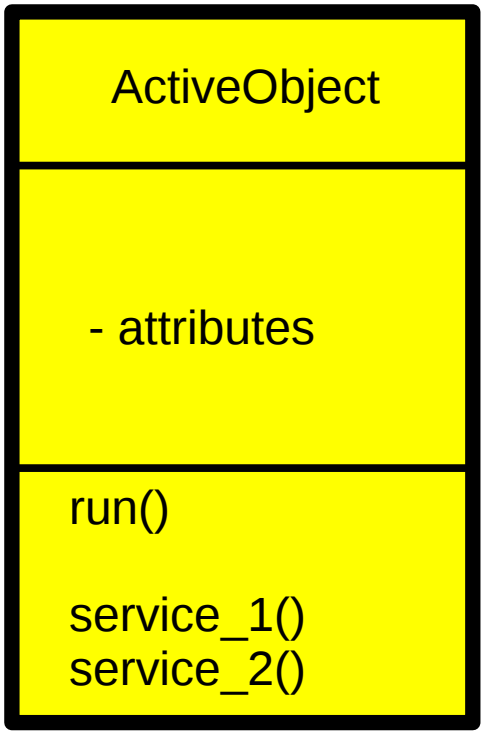
- Introduction, Microcontroller basics
- 32-bit Microcontroller technology
- ARM Cortex M Architecture, (CM0 CM3 CM4 CM7)
- Cortex M Microcontroller Instruction Set
- Exception Handling, Interrupts
- Real-time Operating Systems (RTOS)
- Memory Management, protected memory, MPU
- Implementing Calculations, Data Formats, FPU
- I/O, Device drivers
- Application Design
- Case Studies, Advanced Design Patterns

https://skript.eit.h-da.de/ams/

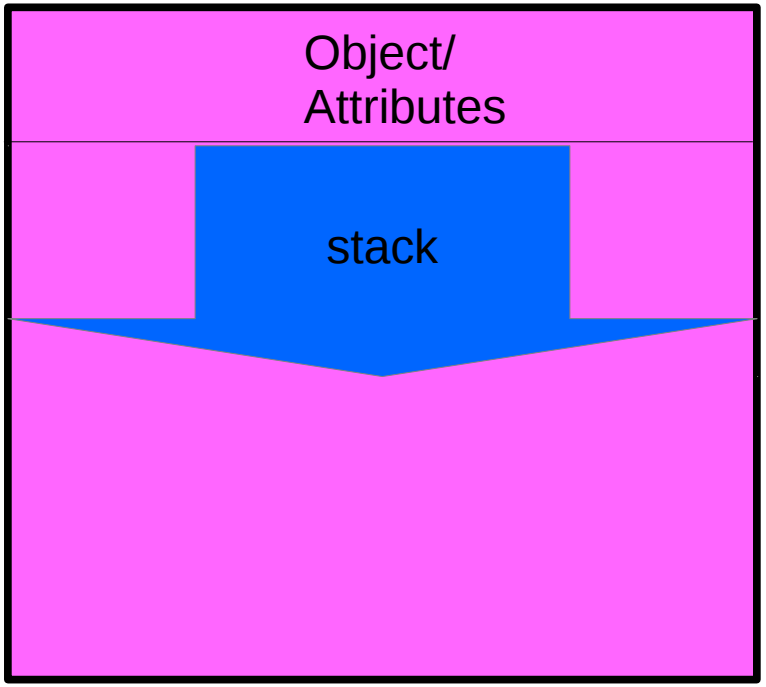
RTOS System: Object Model



Idea: Active Objects



Class Model



top

bottom

Memory Map

Blinker: Active Objects

```
class blinker
{
public:
	blinker( int time)
		: ticker(time)
	{}

	void run( void)
	{
		BSP_LED_Init(LED1);
		for( uint8_t rhythm=0; true;)
		{
			ticker.sync();
			switch( rhythm++)
			{
			case 0:
			case 1:
			case 3:
			case 6:
				BSP_LED_Toggle(LED1);
			break;
			case 8:
				rhythm=1;
				BSP_LED_Toggle(LED1);
			break;
		}
	}
}
private:
	synchronous_timer ticker;
};


active_object <blinker, int> led_blinker( 111, configMINIMAL_STACK_SIZE, STANDARD_TASK_PRIORITY+3,"LED");
```

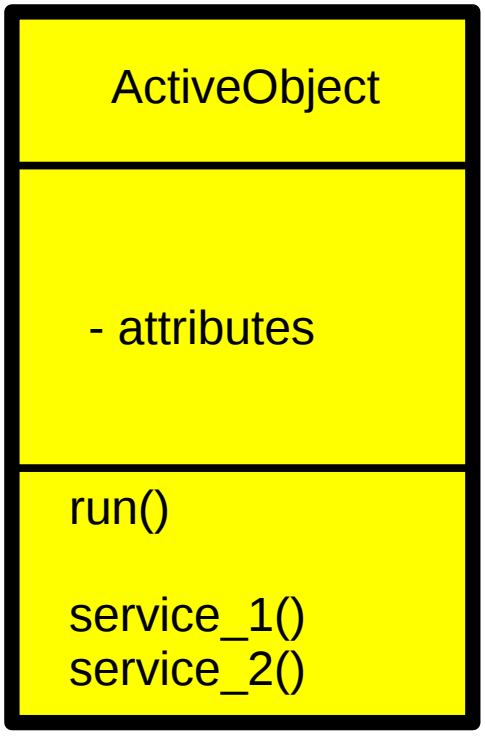
Class ActiveObject

```
//! Object factory for active objects
//! \param runner class implementing a run() method
//! \param info argument type for the constructor of runner
template <class runner, typename info> class active_object
{
typedef struct
{
      info        task_info;        //!< arbitrary pointer for data for the runnable task
      runner **   pp_instance;      //!< address of pointer to the active object
}task_info_block;                         //!< helper structure to provide task start information
public:
      //! constructor for the active object
      //! \param data Pointer to arbitrary data to pass to the runnable task

active_object( info create_info, unsigned stacksize=configMINIMAL_STACK_SIZE,
            UBaseType_t priority=STANDARD_TASK_PRIORITY,
            char const * task_name="A_0");

//! Getter function for the runnable object
inline runner & instance( void) const
{
      return *the_instance;
}
//! Getter function for the executing task
inline const Task & get_task( void) const
{
      return task;
}
private:
      //! this method is run within the created task
      //!
      //! it creates the active object and execute it's run() method
      static void start( task_info_block *data);
      task_info_block init_data;    //!< all information for task start
      runner *the_instance;         //!< pointing to the instance of the active object
      RestrictedTask task;          //!< the task belonging to the active object
};
```

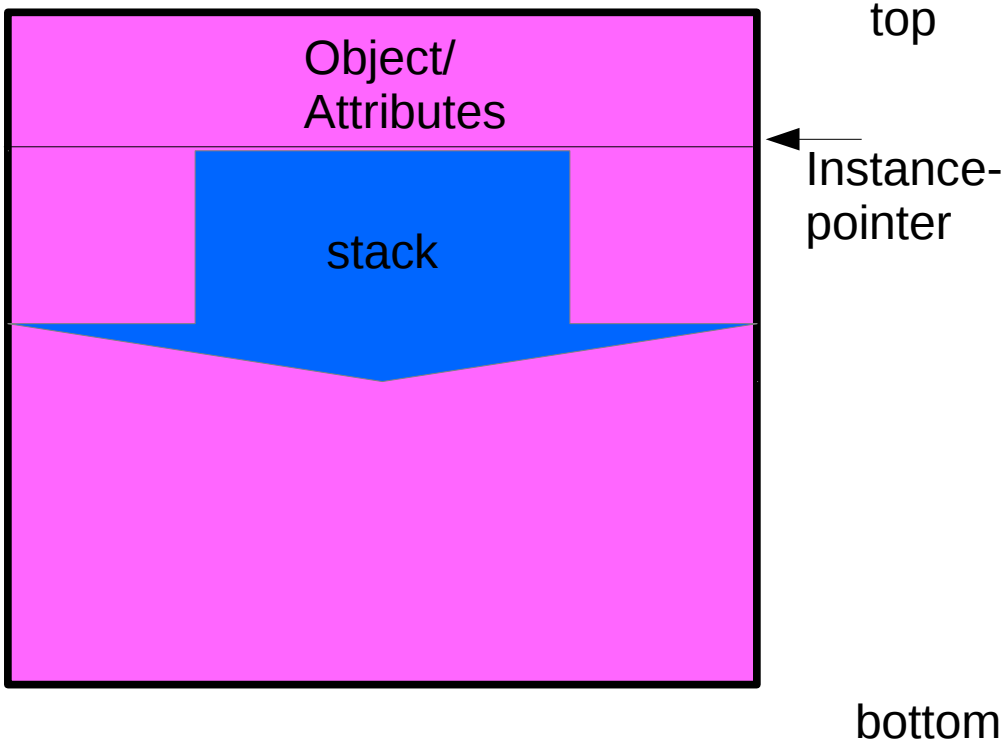
Active Object: Constructor

```
//! Object factory for active objects
//! \param runner class implementing a run() method
//! \param info argument type for the constructor of runner
template <class runner, typename info> class active_object
{
typedef struct
{
      info        task_info;        //!< arbitrary pointer for data for the runnable task
      runner **   pp_instance;      //!< address of pointer to the active object
}task_info_block;                   //!< helper structure to provide task start information
public:
//! constructor for the active object
//! \param data Pointer to arbitrary data to pass to the runnable task

active_object( info create_info, unsigned stacksize=configMINIMAL_STACK_SIZE,
      UBaseType_t priority=STANDARD_TASK_PRIORITY, char const * task_name="A_O")
            :     init_data( { create_info, &the_instance}),

                  the_instance(0),

                  task(
                  {
                        (TaskFunction_t) start,
                        task_name,
                        // max. possible stack fitting into the MPU page in 32bit word units
                        (uint16_t)fit_size( stacksize*sizeof( portSTACK_TYPE) + sizeof(runner)),
                        (void *)&init_data,
                        priority | portPRIVILEGE_BIT,
                        // align the stack to the bottom of our MPU page
                        // the stack top is the bottom of our (*this) object
                        0,
                        {
                              {COMMON_BLOCK,COMMON_SIZE,portMPU_REGION_READ_WRITE},
                              {0,0,0},
                              {0,0,0}
                        }
                  }
                  )
                  {};
```

Active Objects



Class Model



Memory Map

Start: static helper method

```
//! this method is run within the created task
//!
//! it creates the active object and execute it's run() method
static void start( task_info_block *data)
{
      runner the_object( data->task_info);            // create object

      *(data->pp_instance) = &the_object;             // remember instance of the active_object

      drop_privileges();                              // go unprivileged

      the_object.run();                               // go running

      ASSERT( 0);                                     // run() should never return !!

      suspend();                                      // just in case it does anyway ...
}
```

Lecture Overview

- Introduction, Microcontroller basics
- 32-bit Microcontroller technology
- ARM Cortex M Architecture, (CM0 CM3 CM4 CM7)
- Cortex M Microcontroller Instruction Set
- Exception Handling, Interrupts
- Real-time Operating Systems (RTOS)
- Memory Management, protected memory, MPU
- Implementing Calculations, Data Formats, FPU
- I/O, Device drivers
- Application Design
- Case Studies, Advanced Design Patterns

https://skript.eit.h-da.de/ams/